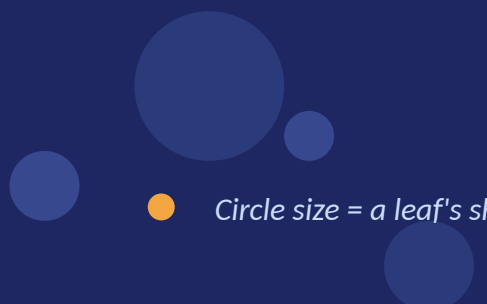




ENSEMBLE LEARNING

LightGBM

Light Gradient Boosting Machine — histogram-based, leaf-wise tree boosting built for scale



● Circle size = a leaf's share of the total loss reduction — the visual motif used throughout this deck

Leaf-wise, Not Level-wise

Most gradient boosting libraries grow a tree level by level. LightGBM instead grows whichever leaf reduces loss the most — across the whole tree.

LEVEL-WISE (typical)



Expands every leaf at the current depth before going deeper — simpler, but wastes splits on low-value leaves.

LEAF-WISE (LightGBM)



Always splits the single leaf with the largest loss reduction — converges faster, for a similar leaf count.

Core Idea

1

Histogram-based splitting

Buckets continuous features into discrete bins, so split search is $O(\text{bins})$ instead of $O(\text{samples})$.

2

GOSS sampling

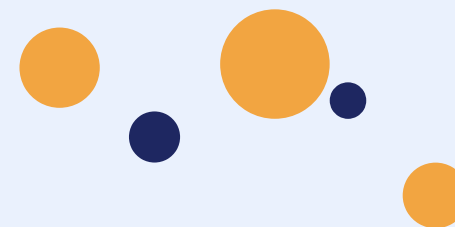
Gradient-based One-Side Sampling keeps high-gradient samples, subsamples the rest — less data, similar accuracy

3

Exclusive Feature Bundling

Sparse, mutually exclusive features are bundled into one, shrinking the feature space.

GOSS: keep big
gradients, sample
the rest



How the Algorithm Runs

1

Bin the features

Continuous values are bucketed into histograms (default 255 bins).

2

Sample with GOSS

Keep the top-gradient rows, randomly sample the rest of the data.

3

Bundle sparse features

Exclusive Feature Bundling merges non-overlapping sparse columns.

4

Find the best leaf split

Scan histograms to find the single split with the largest gain.

5

Grow that leaf (leaf-wise)

Split only the highest-gain leaf, which may unbalance the tree.

6

Shrink, add, repeat

Scale by learning rate η , add the tree, and repeat for n rounds.

The Key Mechanics

The three techniques behind LightGBM's speed, expressed simply

1. Split gain (same as XGBoost)

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma$$

Histograms make scanning every bin for this gain cheap.

2. GOSS sample weight

$$w = 1 \text{ for top-}a \text{ gradients, } (1-a)/b \text{ otherwise}$$

Keeps top a% by gradient, samples b% of the rest, reweights them.

3. Leaf-wise gain priority

`argmax over all current leaves of Gain(leaf)`

Unlike level-wise, the leaf chosen can be anywhere in the tree.

4. Max depth guard

`max_depth` limits leaf-wise growth

Leaf-wise trees can get deep fast — this caps depth to control overfitting.

Why It Scales & Key Hyperparameters

Built for Big Data

Lower memory use via histograms, native support for categorical features, and parallel/distributed training.

These choices make LightGBM noticeably faster than XGBoost on large, high-dimensional datasets, at comparable accuracy.

num_leaves

Main complexity control for leaf-wise trees

max_depth

Optional depth cap to prevent overfitting

learning_rate

Shrinkage η applied to each tree's output

feature_fraction

Fraction of features sampled per tree

Advantages & Limitations

ADVANTAGES

- Much faster training on large, high-dimensional data
- Lower memory footprint via histogram binning
- Native categorical feature support, no one-hot needed

LIMITATIONS

- Leaf-wise growth can overfit small datasets easily
- num_leaves and depth need careful joint tuning
- More sensitive to noisy data than level-wise growth

When to Reach for LightGBM



Large tabular datasets

Millions of rows or high-dimensional features where XGBoost gets slow.



Datasets with categoricals

Native handling avoids costly one-hot encoding pipelines.



Time-boxed experimentation

Faster training means more iterations on features and tuning.



Distributed / GPU training

Built-in support for parallel, distributed, and GPU learning.

Quick Start — lightgbm

```
import lightgbm as lgb

model = lgb.LGBMRegressor(
    n_estimators=300,
    num_leaves=31,
    max_depth=-1,
    learning_rate=0.05,
    feature_fraction=0.8,
    bagging_fraction=0.8,
    random_state=42,
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Notes

- Tune num_leaves before max_depth — it drives complexity here
- Pass categorical columns directly via categorical_feature
- Use early stopping with a validation set to pick n_estimators

Key Takeaways

- 1 Grows trees leaf-wise — always splitting the highest-gain leaf
- 2 Uses histogram binning, GOSS, and EFB to train fast at scale
- 3 Handles categorical features and large data natively
- 4 Powerful but overfit-prone — tune num_leaves and depth together

Thank you